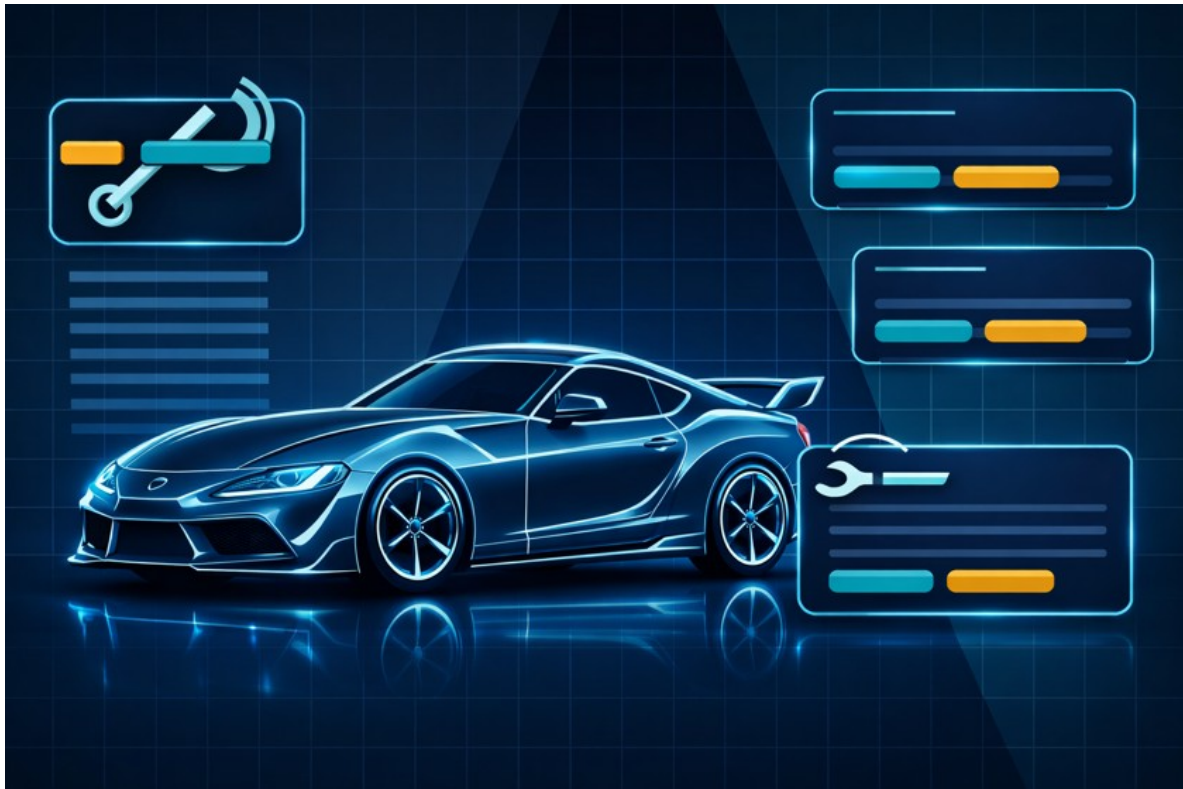


WerkstattPilot

Digitale Auftragsabwicklung für KFZ-Werkstätten



Software Engineering 1 - Projektarbeit

Aufgabe 4: Objektorientiertes Design (OOD)

Schule / Lehrgang	Juventus Schule HF Zürich / HF Informatik Applikationsentwicklung
Modul	Software Engineering 1
Dozent	Arif Chughtai
Projekt	WerkstattPilot
Arbeitsgruppe	WerkstattPilot-Team
Autoren	Livio Luna, Dominic Lolos
Version	1.0
Datum	05.06.2026
Status	Abgabeverision

Projektbild / Profilbild: vom Team bereitgestellte WerkstattPilot-Grafik

Dokumentstatus

Version	Datum	Änderung	Verantwortlich
1.0	05.06.2026	Aufgabe 4 neu erstellt: Business-Schicht, Persistence-Schicht, integriertes 3-Schichtenmodell, Komponentendiagramm, Sequenzdiagramm, Glossar, Dozentenfeedback und Arbeitsprotokoll.	Arbeitsgruppe

Prüfungsabdeckung Aufgabe 4

#	Bewertungsgegenstand	Abdeckung im Dokument
1	Designmodell Business-Schicht (UML Klassendiagramm)	Kapitel 4
2	Designmodell Persistence-Schicht (UML Klassendiagramm)	Kapitel 5
3	Integriertes Designmodell (UML Klassendiagramm)	Kapitel 6.2
4	Integriertes Designmodell (UML Komponentendiagramm)	Kapitel 6.1
5	Normalablauf / Szenario (UML Sequenzdiagramm)	Kapitel 7
6	Glossar der Verantwortlichkeiten	Kapitel 8
7	Rückmeldungen des Dozenten	Kapitel 9
8	Arbeitsprotokoll	Kapitel 10
9	Schreibweise und Konsistenz	Gesamtdokument

Inhaltsverzeichnis

- 1 Ziel und Grundlage des Designmodells
- 2 Umfang, Annahmen und Konsistenzkorrekturen
- 3 Architektur und Package-Struktur
- 4 Designmodell Business-Schicht
- 5 Designmodell Persistence-Schicht
- 6 Integriertes Designmodell
- 7 Sequenzdiagramm UC1 Auftrag anlegen
- 8 Glossar der Design-Verantwortlichkeiten
- 9 Rückmeldungen des Dozenten
- 10 Arbeitsprotokoll

1 Ziel und Grundlage des Designmodells

Dieses Dokument beschreibt das objektorientierte Designmodell für WerkstattPilot. Das Designmodell basiert auf dem Analysemodell aus Aufgabe 3 und auf der Projektumgebung aus Aufgabe 2. Es konkretisiert, wie die erste Iteration technisch in einer 3-Schichtenarchitektur umgesetzt werden soll.

Der Fokus liegt bewusst auf den beiden priorisierten Use Cases der ersten Iteration: UC1 Auftrag anlegen und UC2 Auftragsübersicht mit Statusänderung. Als zentrale fachliche Klasse wird Auftrag verwendet.

Grundlage	Bedeutung für Aufgabe 4
Aufgabe 1 - Projektauftrag	Definiert Problemstellung, Ziele, Stakeholder, priorisierte Use Cases und Rahmenbedingungen.
Aufgabe 2 - Projektumgebung	Legt Werkzeuge, Ordnerstruktur, Java-Packages, Git, Excel Issue Tracking und keine echte Datenbank für Iteration 1 fest.
Aufgabe 3 - Analysemodell	Liefert die fachlichen Klassen, Beziehungen und das Zustandsmodell des Auftrags.
Aufgabe 4 - Aufgabenstellung	Verlangt Designmodelle für Business und Persistence, integriertes 3-Schichtenmodell, Sequenzdiagramm und Glossar.
Aufgabe 5 - Implementation	Bestätigt für später: Java-Prototyp mit Mock-DB, Unit-Tests, ConsoleClient, Javadoc und Endbenutzertest.

2 Umfang, Annahmen und Konsistenzkorrekturen

2.1 Umfang der ersten Iteration

Use Case	Design-Relevanz	Umsetzung im Designmodell
UC1 Auftrag anlegen	Auftrag mit Kundendaten, Fahrzeugkennzeichen, Beanstandung und Termin erstellen.	ConsoleClient ruft AuftragService auf. AuftragServiceImpl erstellt über AuftragFactory ein Auftrag-Objekt und speichert es über AuftragRepository.
UC2 Auftragsübersicht & Status	Aufträge lesen, auf der Konsole anzeigen und den Status ändern.	AuftragService bietet Methoden zum Lesen aller Aufträge und zum Ändern des Auftragsstatus.

2.2 Annahmen

Annahme	Begründung
Zentrale fachliche Klasse ist Auftrag.	Auftrag verbindet die Auftragserfassung, die Übersicht und den Statuswechsel.
Die erste Implementation speichert nur die für den Prototyp nötigen Werte.	Aufgabe 5 verlangt eine kleine, testbare Implementation und keine vollständige Fachanwendung.
Es gibt keine Benutzereingaben über die Tastatur.	Der spätere ConsoleClient kann mit hart implementierten Testdaten arbeiten.
Eine echte Datenbank wird nicht verwendet.	Die Persistence-Schicht arbeitet mit einer Mock-DB im Hauptspeicher.
Das Design zeigt keine Attribute.	Die Aufgabenstellung zu Aufgabe 4 verlangt Klassen und Schnittstellen, jedoch keine Attribute.
Methoden werden nur bei Schnittstellen gezeigt.	Dadurch bleibt das Design schlank und entspricht den Vorgaben der Aufgabe.

2.3 Konsistenzkorrektur: MariaDB ausgebaut

In einer früheren Projektidee war zeitweise eine echte lokale Datenbank bzw. MariaDB diskutiert. Diese Variante wurde für Aufgabe 4 bewusst entfernt. Das Designmodell verwendet keine MariaDB, kein JDBC und keine SQL-Abfragen. Die Datenhaltung erfolgt über AuftragRepositoryMock als In-Memory Mock-DB.

Damit ist das Design konsistent mit der Projektumgebung aus Aufgabe 2 und mit den Hinweisen für die spätere Implementation in Aufgabe 5.

3 Architektur und Package-Struktur

Die Architektur folgt der vorgegebenen logischen 3-Schichtenstruktur. Die Packages orientieren sich an der in Aufgabe 2 dokumentierten Ordnerstruktur.

```
WerkstattPilot/  
  03_entwicklung/  
    werkstattpilot/  
      src/main/java/ch/werkstattteam/werkstattpilot/  
        presentation/  
        business/  
        persistence/  
      src/test/java/ch/werkstattteam/werkstattpilot/test/  
        business/  
        persistence/
```

3.1 Package-Übersicht

Basispackage: *ch.werkstattteam.werkstattpilot*

Schicht	Package	Verantwortung
Presentation	.presentation	Start der Anwendung, ConsoleClient, Anzeige von Testdaten und Resultaten auf der Konsole.
Business	.business	Fachliche Objekte, Services, Statuslogik und Factories für Aufträge.
Persistence	.persistence	Repository-Schnittstelle, Mock-Repository und gekapselte In-Memory-Datenhaltung.
Tests Business	.test.business	JUnit-Tests für Business-Logik in Aufgabe 5.
Tests Persistence	.test.persistence	JUnit-Tests für Repository und Mock-DB in Aufgabe 5.

3.2 Technische Abhängigkeiten

Abhängigkeit	Regel
Presentation -> Business	ConsoleClient verwendet AuftragService bzw. AuftragServiceFactory.
Business -> Persistence	AuftragServiceImpl verwendet AuftragRepository zum Speichern und Lesen von Aufträgen.
Persistence -> Business	AuftragRepositoryMock speichert Auftrag-Objekte aus der Business-Schicht.
Keine direkte Presentation -> Persistence Abhängigkeit	Die Konsole greift nicht direkt auf RepositoryMock zu.
Keine echte DB	Persistence nutzt nur eine Collection im Hauptspeicher.

4 Designmodell Business-Schicht

Die Business-Schicht enthält die fachliche Koordination für die Use Cases der ersten Iteration. Der wichtigste Einstiegspunkt ist die Schnittstelle AuftragService. Die konkrete Klasse AuftragServiceImpl setzt die fachliche Logik um und verwendet Factory und Repository.

4.1 Schnittstelle AuftragService

Methode	Rückgabe	Parameter	Zweck
auftragAnlegen	Auftrag	kundenName: String, kennzeichen: String, beschreibung: String, termin: LocalDate	Erstellt einen neuen Auftrag und speichert ihn über das Repository.
alleAuftraegeAnzeigen	List<Auftrag>	-	Liest alle gespeicherten Aufträge für die Auftragsübersicht.
statusAendern	Auftrag	auftragsNummer: int, neuerStatus: Auftragsstatus	Ändert den Status eines bestehenden Auftrags und aktualisiert ihn im Repository.

4.2 UML Klassendiagramm Business-Schicht

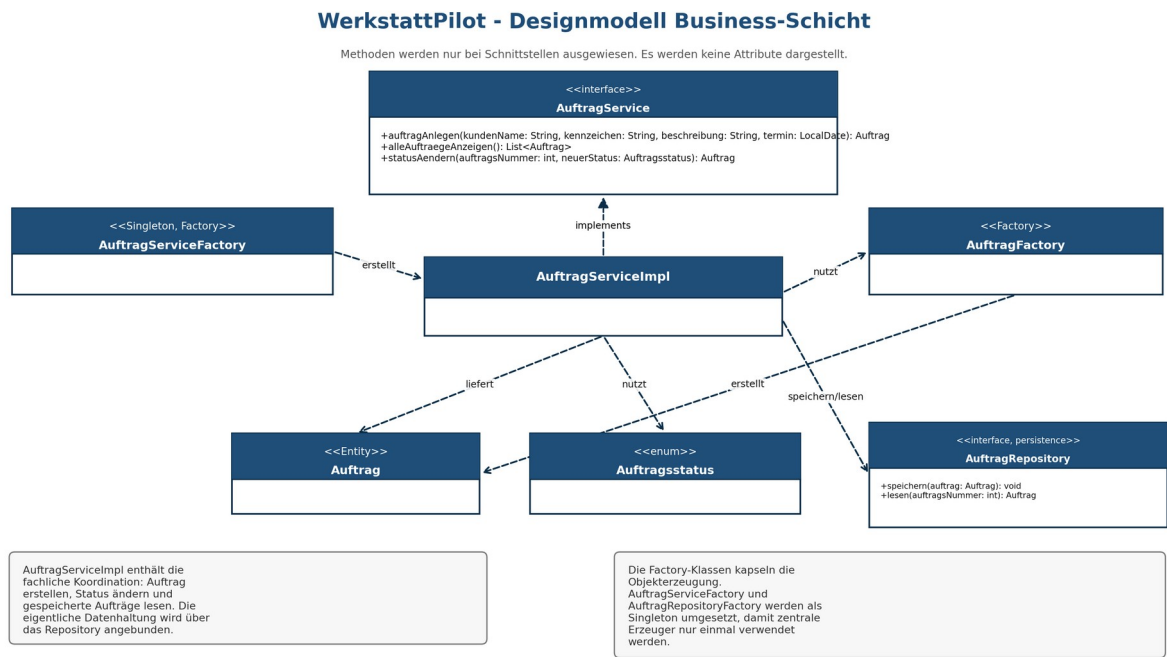


Abbildung 1: Business-Schicht mit AuftragService, Service-Implementierung, Factories und fachlichen Klassen.

4.3 Eingesetzte Patterns in der Business-Schicht

Pattern	Klasse / Schnittstelle	Begründung
Factory	AuftragFactory	Erzeugt Auftrag-Objekte zentral, damit die Objekterstellung nicht im ConsoleClient verteilt wird.
Factory	AuftragServiceFactory	Erzeugt bzw. liefert den Business-Service zentral.
Singleton	AuftragServiceFactory	Die Service-Erzeugung wird zentral gehalten und nicht mehrfach im Code verteilt.
Repository	AuftragRepository	Kapselt den Datenzugriff für Auftrag-Objekte hinter einer Schnittstelle.

5 Designmodell Persistence-Schicht

Die Persistence-Schicht kapselt das Speichern und Lesen von Auftrag-Objekten. Für die erste Iteration wird keine echte Datenbank verwendet. Das RepositoryMock speichert die Aufträge im Hauptspeicher mit einer Java Collection.

5.1 Schnittstelle AuftragRepository

Methode	Rückgabe	Parameter	Zweck
speichern	void	auftrag: Auftrag	Speichert einen neuen Auftrag in der Mock-DB.
lesen	Auftrag	auftragsNummer: int	Liest einen einzelnen Auftrag anhand der Auftragsnummer.
alleLesen	List<Auftrag>	-	Liest alle gespeicherten Aufträge.
aktualisieren	void	auftrag: Auftrag	Aktualisiert einen bestehenden Auftrag, zum Beispiel nach Statuswechsel.

5.2 UML Klassendiagramm Persistence-Schicht

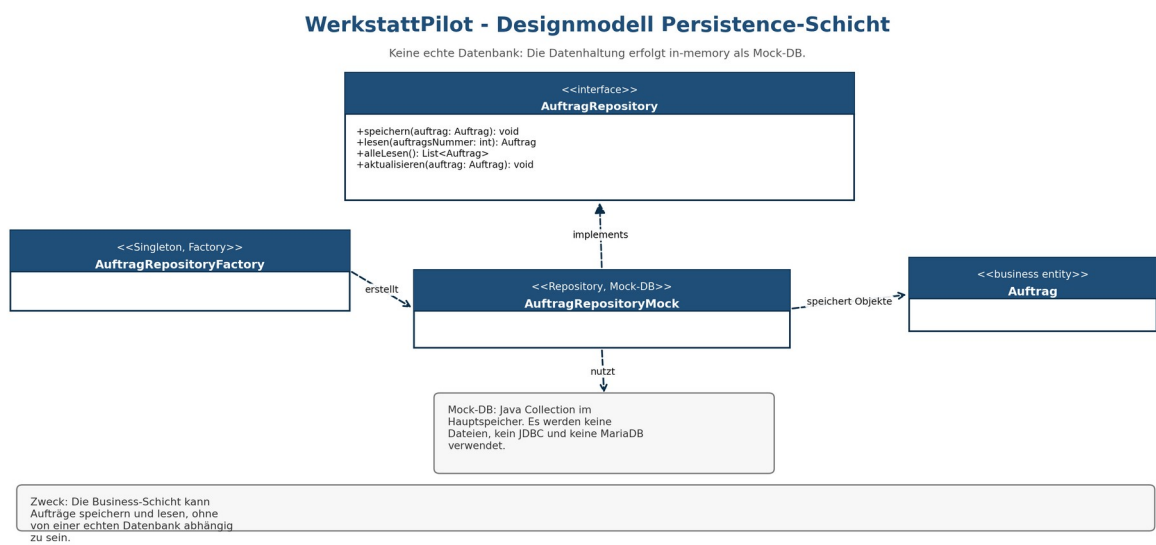


Abbildung 2: Persistence-Schicht mit Repository-Schnittstelle, Mock-Repository und Factory.

5.3 Datenhaltung ohne MariaDB

AuftragRepositoryMock hält die Aufträge während der Programmausführung im Hauptspeicher. Dadurch kann die Persistence-Schicht in Aufgabe 5 mit Unit-Tests geprüft werden, ohne dass Datenbankinstallation, SQL-Schema oder externe Infrastruktur benötigt werden.

Nicht verwendet	Stattdessen verwendet
MariaDB	AuftragRepositoryMock
JDBC-Verbindung	Java Collection im Hauptspeicher
SQL-Tabellen	Objekte vom Typ Auftrag
Dateisystem-Speicherung	In-Memory Mock-DB

6 Integriertes Designmodell

Das integrierte Designmodell zeigt, wie Presentation, Business und Persistence zusammenspielen. Die Presentation-Schicht kennt nur die Business-Schicht. Die Business-Schicht koordiniert den Use Case und greift über Repository-Schnittstellen auf die Persistence-Schicht zu.

6.1 UML Komponentendiagramm

WerkstattPilot - UML Komponentendiagramm 3-Schichtenmodell

Technische Abhängigkeiten: Presentation ruft Business auf, Business ruft Persistence auf.

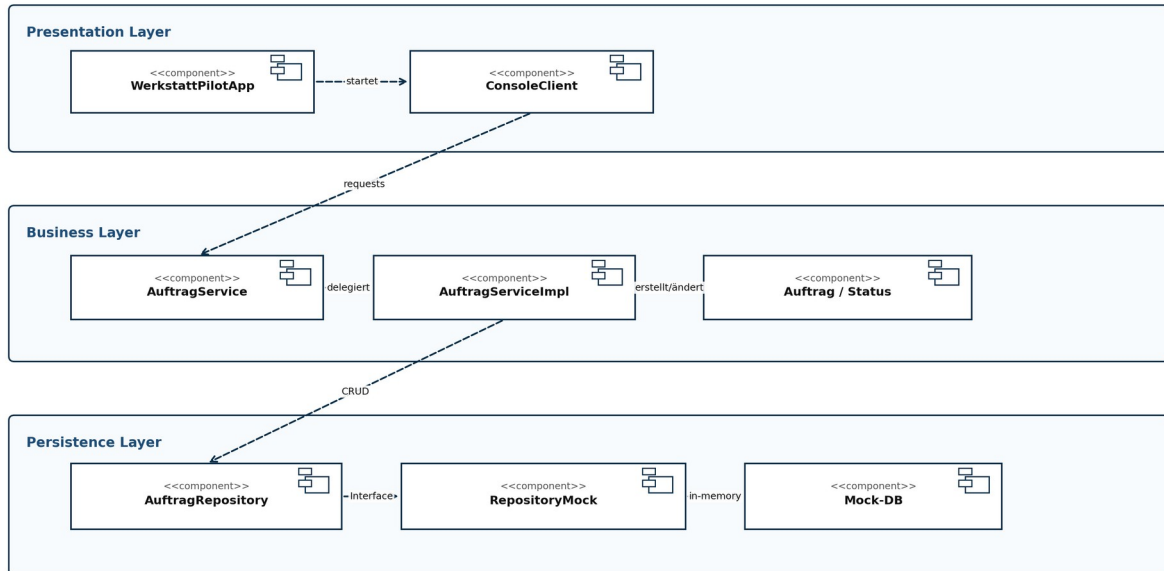


Abbildung 3: Komponentendiagramm mit technischer Abhängigkeit von Presentation zu Business und von Business zu Persistence.

6.2 UML Klassendiagramm integriert

WerkstattPilot - Integriertes Klassendiagramm

Packages entsprechen der Ordnerstruktur aus Aufgabe 2: presentation, business, persistence.

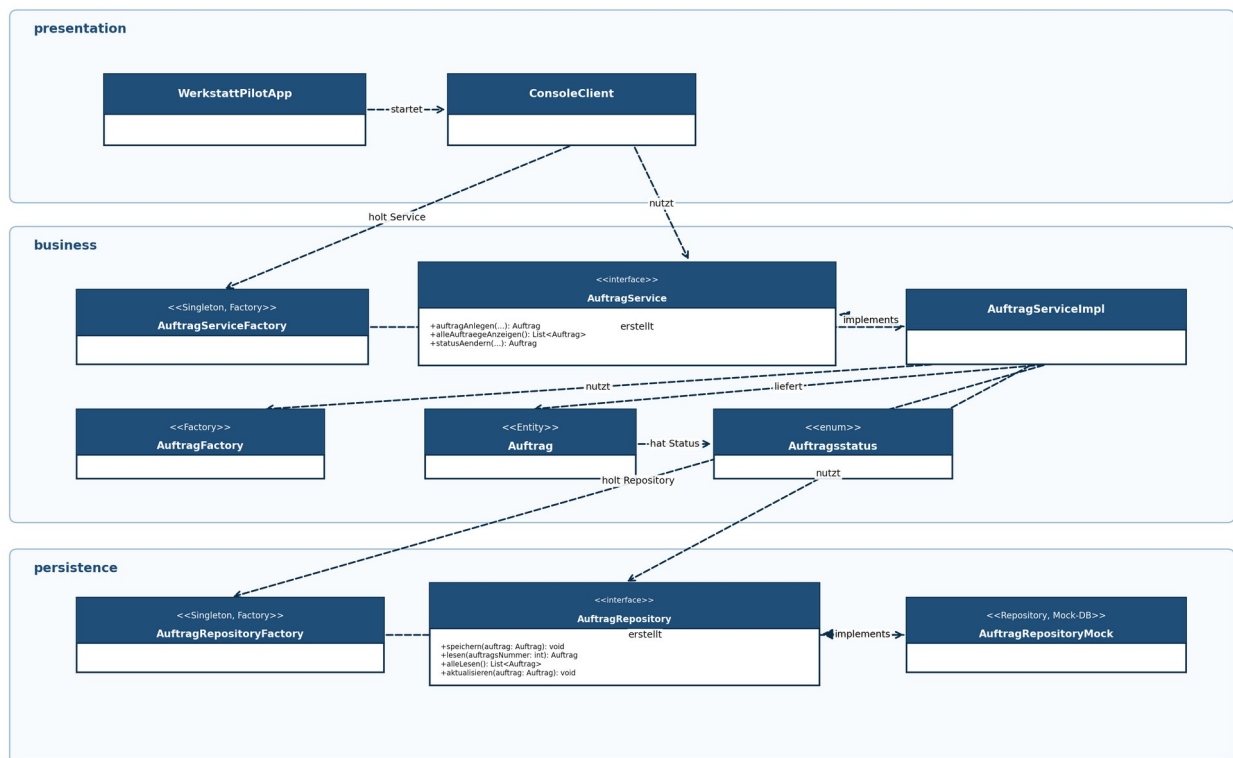


Abbildung 4: Integriertes Klassendiagramm der drei Packages presentation, business und persistence.

6.3 Vollständigkeitsprüfung des integrierten Modells

Prüfrage	Ergebnis
Kann die Anwendung gestartet werden?	Ja. WerkstattPilotApp verwendet ConsoleClient.
Kann UC1 Auftrag anlegen ausgeführt werden?	Ja. ConsoleClient ruft AuftragService auf. AuftragServiceImpl erstellt und speichert einen Auftrag.
Kann UC2 Auftragsübersicht und Statuswechsel ausgeführt werden?	Ja. AuftragService bietet Lesen aller Aufträge und statusAendern an.
Ist die Datenhaltung gekapselt?	Ja. Zugriff erfolgt über AuftragRepository. Die konkrete Mock-Implementierung bleibt austauschbar.
Sind die Patterns sichtbar?	Ja. Factory, Singleton und Repository sind im Modell gekennzeichnet.
Ist das Modell konsistent mit Aufgabe 2?	Ja. Packages und Ordner entsprechen presentation, business und persistence. MariaDB ist entfernt.

7 Sequenzdiagramm UC1 Auftrag anlegen

Das Sequenzdiagramm überprüft den Normalablauf des Use Case UC1 Auftrag anlegen. Es zeigt, dass ConsoleClient, AuftragService, AuftragFactory, AuftragRepository und Mock-DB sauber zusammenarbeiten.

Sequenzdiagramm UC1: Auftrag anlegen

Normalablauf mit AuftragService, Factory und RepositoryMock

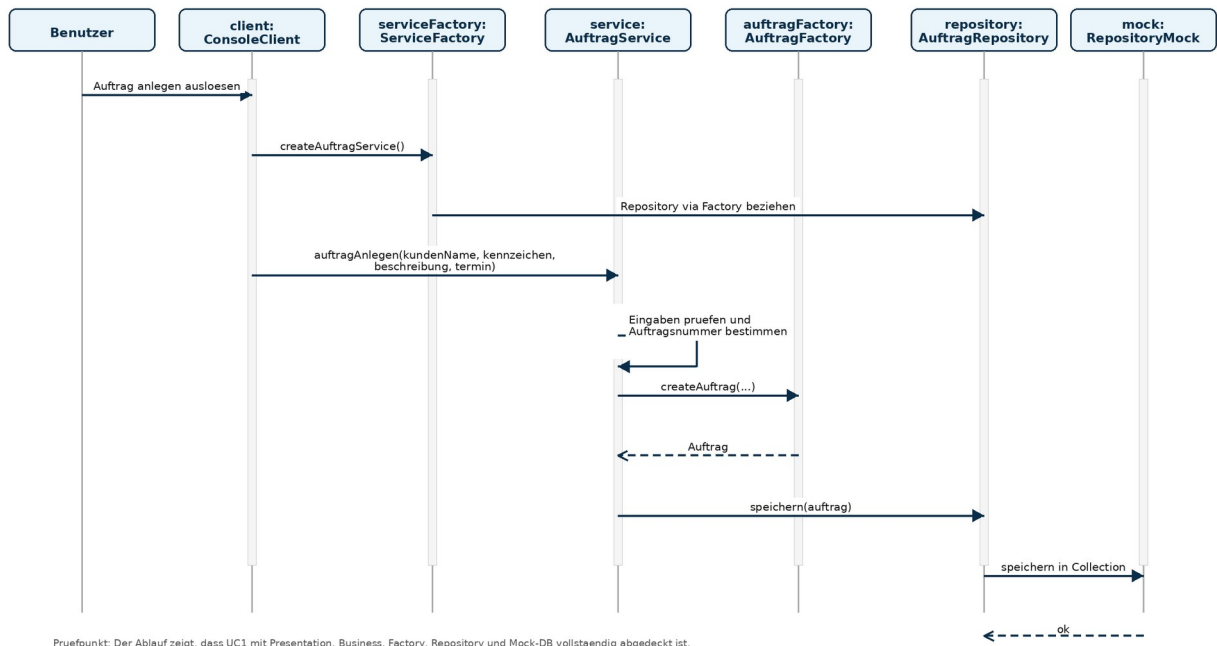


Abbildung 5: Sequenzdiagramm für den Normalablauf von UC1 Auftrag anlegen.

Prüfpunkt	Ergebnis
Fachliche Koordination	AuftragService koordiniert die Erstellung und Speicherung des Auftrags.
Objekterstellung	AuftragFactory erzeugt das fachliche Auftrag-Objekt zentral.
Speicherung	AuftragRepository kapselt den Datenzugriff. AuftragRepositoryMock speichert in der Mock-DB.
Konsistenz	Der Ablauf benötigt keine MariaDB und passt zur Package-Struktur aus Aufgabe 2.

8 Glossar der Design-Verantwortlichkeiten

Das Glossar beschreibt die Verantwortlichkeiten jeder Klasse und Schnittstelle im Designmodell. Es erklärt nicht die UML-Notation, sondern den konkreten Zweck im Projekt WerkstattPilot.

Element	Schicht	Typ	Verantwortlichkeit
WerkstattPilotApp	Presentation	Klasse	Startet die Anwendung und verwendet ein ConsoleClient-Objekt für den Endbenutzertest.
ConsoleClient	Presentation	Klasse	Erstellt Testdaten, ruft die Business-Schicht auf und gibt Aufträge auf der Konsole aus.
AuftragService	Business	Schnittstelle	Definiert die fachlichen Operationen für Auftrag anlegen, Aufträge anzeigen und Status ändern.
AuftragServiceImpl	Business	Klasse	Setzt die fachlichen Operationen um und koordiniert Factory und Repository.
AuftragServiceFactory	Business	Singleton / Factory	Liefert zentral eine AuftragService-Instanz und kapselt die Service-Erzeugung.
AuftragFactory	Business	Factory	Erzeugt Auftrag-Objekte an einer zentralen Stelle.
Auftrag	Business	Fachliche Klasse	Repräsentiert den Werkstattauftrag der ersten Iteration.
Auftragsstatus	Business	Enum / Wertetyp	Definiert die erlaubten Zustände eines Auftrags: Offen, In Arbeit, Wartet auf Teile, Fertig, Storniert.
AuftragRepository	Persistence	Schnittstelle / Repository	Definiert die Datenzugriffsoperationen für Speichern, Lesen, alle Lesen und Aktualisieren von Aufträgen.
AuftragRepositoryMock	Persistence	Repository-Implementierung	Speichert Auftrag-Objekte im Hauptspeicher und ersetzt eine echte Datenbank.
AuftragRepositoryFactory	Persistence	Singleton / Factory	Liefert zentral eine Repository-Instanz und kapselt die Wahl der konkreten Persistence-Implementierung.

9 Rückmeldungen des Dozenten

Datum	Rückmeldung	Umsetzung im Dokument	Status
Di, 21.04.2026	Der Dozent hat bestätigt, dass das Projekt auf einem guten Weg ist.	Das Designmodell bleibt auf die erste Iteration fokussiert und wird nicht unnötig erweitert.	berücksichtigt
Di, 05.05.2026	Im Grossen und Ganzen gut. Einzelne kleine Anpassungen sind noch offen, insbesondere bei Konsistenzen.	MariaDB wurde aus dem Design entfernt. Die Persistence-Schicht ist nun konsistent als Mock-DB / In-Memory Repository modelliert. Packages und Ordnerstruktur wurden mit Aufgabe 2 abgeglichen.	umgesetzt

10 Arbeitsprotokoll

Datum	Beteiligte	Tätigkeit	Ergebnis	Nächster Schritt
05.05.2026	Livio Luna, Dominic Lolos, Dozent	Zwischenbesprechung Aufgabe 4 und Konsistenzprüfung.	Grundrichtung bestätigt; kleine Konsistenzanpassungen erkannt.	MariaDB-Bezug entfernen und Design schärfen.
05.06.2026	Livio Luna	Aufgabe 4 neu strukturiert.	Dokumentaufbau gemäss Bewertungskriterien erstellt.	Diagramme und Glossar finalisieren.
05.06.2026	Livio Luna	Business- und Persistence-Modell erstellt.	AuftragService, AuftragRepository, Factories und Mock-Repository definiert.	Integriertes Modell prüfen.
05.06.2026	Livio Luna	Komponentendiagramm und Sequenzdiagramm erstellt.	3-Schichtenmodell und UC1 Normalablauf dokumentiert.	DOCX und PDF finalisieren.

11 Quellen und Referenzen

Quelle	Verwendung im Projekt
Aufgabenblatt Aufgabe 4	Vorgaben für Business-Schicht, Persistence-Schicht, integriertes Designmodell, Komponentendiagramm, Sequenzdiagramm, Patterns und Glossar.
Aufgabe 2 und Aufgabe 3	Grundlage für Ordnerstruktur, Packages, Mock-DB-Entscheid und zentrale fachliche Klasse Auftrag.
Aufgabenblatt Aufgabe 5	Vorgaben für spätere Implementation mit Mock-DB, Unit-Tests, ConsoleClient und Javadoc.